cf578ad3-f012-4c45-a7f3-c4dec45b508a

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'sillsdev/TheCombine' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/TheCombine/TheCombine (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-02-26T22:24:13.878Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Filter and order glosses/definitions by project analysis languages
</pr_title>

<pr_description>
In Review Entries and MergeDragDrop, glosses and definitions were shown for all languages in storage order, ignoring the project's configured analysis writing systems.

Changes

- **`FontContext` (`fontContext.ts`)**: Added `analysisLangs: string[]` to `ProjectFonts`, populated from `proj.analysisWritingSystems` in order. Makes analysis language

ordering available to any component via context without additional Redux selectors.

- **`SensesTextSummary`** (backing `GlossesCell` and `DefinitionsCell` in ReviewEntriesTable): Reads `analysisLangs` from `FontContext` and filters/orders glosses and definitions accordingly. Falls back to show-all behavior when `analysisLangs` is empty.

- **`SidebarDragSense`**: Now passes `languages={analysisLangs}` (from `FontContext`) to `SenseCardContent`, consistent with `DragSense` which already did this via Redux.

- **Tests**: Added assertions to `fontContext.test.ts` covering the new `analysisLangs` property.

```tsx
// SensesTextSummary now respects analysis language order
const analysisLangs = useContext(FontContext).analysisLangs;
// ...
texts = analysisLangs.flatMap((l) =>
  sense.glosses.filter((g) => g.language === l).map((g) =>
g.def.trim())
);
```

<issue_title>Display of glosses/definitions not affected by project analysis langs</issue_title>
><issue_description>In Review Entries (and possibly elsewhere), the gloss cell is a concatenation of all glosses, grouped by language, but otherwise appearing in the order they are stored in the word. (Likewise for the definitions.) Ideally:
>
> - They would show up in the same order as the project analysis languages
> - Any in other languages wouldn't show up at all
>
> Q: Should this be handled in the frontend? Or should words fetched from the database have the content sorted in the backend according to the project settings?
> A: Probably in the frontend, so various things that fetch the whole frontier aren't slowed down. In the frontend, we can modify and use the FontContext to get the analysis language order.
>
> TODO:
>

> - Modify `FontContext` to hold the list of language tags for project analysis languages
> - In `ReviewEntriesTable`: modify the gloss cell and definition cell to only show the content that matches analysis languages, and in the same order
> - In `MergeDragDrop`: modify display of glosses and definitions to only show the content that matches analysis languages, and in the same order</issue_description>
>
> ## Comments on the Issue (you are @copilot in this section)
>
><comments>
></comments>
></pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_new>
<comment_id>3863860284</comment_id>
<author>@imnasnainaec</author>
@copilot

* Revert the changes to `fontContext`
* Remove the `languages` prop from both `SenseCardContent` and `SenseCardText`
* Move the `analysisLangs = useAppSelector` out of `DragSense` and into both `SenseCardText` and `SensesTextSummary`
* Update tests accordingly
* Revert the changes to `package-lock.json`
</comment_new>

</pr_comments>

</comments>

The last **2** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>

```
  * Start by fully understanding <problem_statement> and the
  <comments> before making any changes. You will only address the
  <comment_new> comments. Reply to new comments as you address them.
  Ignore comments that are not actionable or directed at you.
  * For each new comment, determine:
      - If it a request for changes, a question on the code or a
  suggestion.
      - Is it addressed to you or someone else, e.g. "Hey
  @username, ..." or "@username can you ...".
      - Is it a general comment or praise.
  * If there are no comments you need to address or reply to, you can
  stop now without exploring or explaining yourself.
  * If the user's ask is not clear, you can reply to the comment and
  ask for clarification.
  * If an action or explanation is needed, follow the steps below.
  </rules_for_making_changes>
```

<prev_assistant>

```
Here's a summary of what was done in response to the reviewer's
feedback:

## Changes Made (commit `9177e4b`)

**Reverted files** (back to original state):
- `src/utilities/fontContext.ts` - no `analysisLangs` property
- `src/utilities/tests/fontContext.test.ts` - original tests
- `src/goals/MergeDuplicates/MergeDupsStep/MergeDragDrop/
SidebarDragSense.tsx` - original
- `package-lock.json` - original

**Modified files** (new approach):
- **`SenseCardText.tsx`**: Removed `languages` prop, now uses
`useAppSelector` with `shallowEqual` to get analysis languages directly
from Redux state
- **`SensesTextSummary.tsx`**: Replaced `useContext(FontContext)` with
`useAppSelector` + `shallowEqual` to get analysis languages from Redux
state
- **`SenseCardContent.tsx`**: Removed the `languages?: string[]` prop
entirely
- **`SenseCard.tsx`**: Removed the `languages?: string[]` prop entirely
```

- **`WordCard/index.tsx`**: Removed the `languages?: string[]` prop entirely
- **`DragSense.tsx`**: Removed the `analysisLangs = useAppSelector(...)` selector and `languages={analysisLangs}` prop from `SenseCardContent`

Updated tests:
- `EditSensesCardContent.test.tsx`: Added Redux `Provider` wrapper since `SensesTextSummary` (used via `SummarySenseCard`) now requires Redux context

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'sillsdev/TheCombine' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/TheCombine/TheCombine (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-02-26T22:43:41.058Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Filter and order glosses/definitions by project analysis languages
</pr_title>

<pr_description>
In Review Entries and MergeDragDrop, glosses and definitions were shown for all languages in storage order, ignoring the project's configured analysis writing systems.

Changes

- **`SensesTextSummary`** (backing `GlossesCell` and `DefinitionsCell` in ReviewEntriesTable): Uses `useAppSelector` to read `analysisLangs` directly from Redux state, then filters and orders glosses/definitions accordingly. Falls back to show-all behavior when `analysisLangs` is empty.

- **`SenseCardText`** (used in MergeDragDrop sense cards): Uses `useAppSelector` to read `analysisLangs` directly from Redux state, then displays only glosses/definitions matching those languages in order. Removed the `languages` prop — language filtering is now handled internally.

- **`SenseCardContent`**, **`SenseCard`**, **`WordCard`**: Removed the now-unnecessary `languages` prop.

- **`DragSense`**: Removed the `analysisLangs` selector and the `languages` prop passed to `SenseCardContent`, since `SenseCardText` handles this internally.

```tsx
// SenseCardText and SensesTextSummary now fetch analysis languages from Redux
const analysisLangs = useAppSelector(
  (state: StoreState) =>

state.currentProjectState.project.analysisWritingSystems.map(
      (ws) => ws.bcp47
    ),
  shallowEqual
);
```

<issue_title>Display of glosses/definitions not affected by project analysis langs</issue_title>
><issue_description>In Review Entries (and possibly elsewhere), the gloss cell is a concatenation of all glosses, grouped by language, but otherwise appearing in the order they are stored in the word. (Likewise for the definitions.) Ideally:
>
> - They would show up in the same order as the project analysis languages
> - Any in other languages wouldn't show up at all
>
> Q: Should this be handled in the frontend? Or should words fetched from the database have the content sorted in the backend according to the project settings?
> A: Probably in the frontend, so various things that fetch

the whole frontier aren't slowed down. In the frontend, we can modify and use the FontContext to get the analysis language order.
>
> TODO:
>
> - Modify `FontContext` to hold the list of language tags for project analysis languages
> - In `ReviewEntriesTable`: modify the gloss cell and definition cell to only show the content that matches analysis languages, and in the same order
> - In `MergeDragDrop`: modify display of glosses and definitions to only show the content that matches analysis languages, and in the same order</issue_description>
>
> ## Comments on the Issue (you are @copilot in this section)
>
><comments>
></comments>
></pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@imnasnainaec</author>
@copilot

* Revert the changes to `fontContext`
* Remove the `languages` prop from both `SenseCardContent` and `SenseCardText`
* Move the `analysisLangs = useAppSelector` out of `DragSense` and into both `SenseCardText` and `SensesTextSummary`
* Update tests accordingly
* Revert the changes to `package-lock.json`
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot
>
> * Revert the changes to `fontContext`...

Done in 9177e4b. Here's a summary of the changes:

- Reverted `fontContext.ts`, `fontContext.test.ts`, `package-lock.json`, and `SidebarDragSense.tsx`
- `SenseCardText` and `SensesTextSummary` now use `useAppSelector` (with `shallowEqual`) to get `analysisLangs` directly from Redux state
- Removed the `languages` prop from `SenseCardContent`, `SenseCardText`, `SenseCard`, and `WordCard`
- Removed the `analysisLangs` selector from `DragSense` (no longer needed since `SenseCardText` handles it internally)
- Updated `EditSensesCardContent.test.tsx` to wrap with a Redux `Provider` (needed since `SensesTextSummary` now uses Redux)
</comment_old>

<comment_new>
<comment_id>3863930791</comment_id>
<author>@imnasnainaec</author>
@copilot If appropriate, replace the other use of `arraysEqual` with `shallowEqual`and delete `arraysEqual`.
</comment_new>

</pr_comments>

</comments>

The last **3** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.

```
   * If an action or explanation is needed, follow the steps below.
   </rules_for_making_changes>
```